

Lab 09

L0901 ให้นักศึกษาเขียนโปรแกรมเพื่อทดลองการสร้างและใช้งาน Pointer เบื้องต้น

กำหนดให้มีตัวแปร 3 ตัว ดังนี้

- ตัวแปร apple เป็นตัวแปรเก็บข้อมูลเลขจำนวนเต็ม มีค่าเริ่มต้นเป็น 17
- ตัวแปร meter เป็นตัวแปรเก็บข้อมูลเลขทศนิยม มีค่าเริ่มต้นเป็น 22.38
- ตัวแปร order เป็นตัวแปรเก็บข้อมูลตัวอักษร 1 ตัว มีค่าเริ่มต้นเป็น 'k'

ให้นักศึกษาทำการสร้างตัวแปร Pointer เพื่อทำการเก็บเลข Address ของตัวแปรทั้ง 3 ตัว อย่างเหมาะสม โดยชื่อของ Pointer นักศึกษาสามารถตั้งชื่อได้อย่างอิสระ

จากนั้นให้นักศึกษาพิมพ์ค่าและเลข Address ของตัวแปรทั้ง 3 ตัวด้วย Pointer นักศึกษาสามารถใช้ %p เพื่อช่วยในการพิมพ์เลข Address ของตัวแปร

ตัวอย่างผล

```
apple is 17, stored at 000000000061FE04
meter is 22.38, stored at 000000000061FE00
order is k, stored at 000000000061FDFF
```

ผลลัพธ์

```
#include <stdio.h>

int main()
{
    int apple = 12;
    float meter = 22.38;
    char order = 'k';

    int *ptr_apple;
    float *ptr_meter;
    char *ptr_order;

    ptr_apple = &apple;
    ptr_meter = &meter;
    ptr_order = &order;

    printf("apple is %d, stored at %p\n", apple, ptr_apple);
    printf("meter is %.2f, stored at %p\n", meter, ptr_meter);
    printf("order is %c, stored at %p", order, ptr_order);
    return 0;
}
```

```
apple is 12, stored at 000000B88EFFF824
meter is 22.38, stored at 000000B88EFFF820
order is k, stored at 000000B88EFFF81F
```

L0902 ให้นักศึกษาเขียนโปรแกรมเพื่อทดลองการสร้างและใช้งาน Pointer เบื้องต้น

กำหนดให้มีตัวแปร 2 ตัวสำหรับเก็บข้อมูลเลขจำนวนเต็ม ได้แก่

- ตัวแปร melon มีค่าเริ่มต้นเป็น 21
- ตัวแปร banana มีค่าเริ่มต้นเป็น 14

ให้นักศึกษาสร้างตัวแปร Pointer ขึ้นมา 2 ตัว โดยให้ Pointer ทั้ง 2 ตัวชี้ไปที่ melon ทั้งคู่ โดย Pointer ตัวหนึ่งจะทำการชี้ไปที่ melon ด้วยการเก็บ Address ของ melon โดยตรง และ Pointer อีกตัวจะทำการชี้ไปที่ melon ด้วยการ Assign มาจาก Pointer ตัวแรก

Step 1: ทำการพิมพ์ค่าที่อยู่ใน Address ที่ Pointer ทั้ง 2 ตัวชี้อยู่

Step 2: เปลี่ยนค่าที่ถูกเก็บใน Address ที่ Pointer ตัวแรกชี้ ให้กลายเป็น 77 แล้ว ทำการพิมพ์ค่าที่อยู่ใน Address ที่ Pointer ทั้ง 2 ตัวชี้อยู่

Step 3: ให้ Pointer ตัวที่ 2 ชี้ไปที่ Banana แล้วพิมพ์ค่าที่อยู่ใน Address ที่ Pointer ทั้ง 2 ตัวชี้อยู่

Step 4: เปลี่ยนค่าที่ถูกเก็บใน Address ที่ Pointer ตัวที่สองชี้ ให้มีค่าเท่ากับค่าเริ่มต้นของ melon (Assign ค่าเข้าเป็นตัวเลขโดยตรงได้) แล้วพิมพ์ค่าที่อยู่ใน Address ที่ Pointer ทั้ง 2 ตัวชี้อยู่

ตัวอย่างผล

```
step 1
pointer1 has 21 in its stored address
pointer2 has 21 in its stored address

step 2
pointer1 has 77 in its stored address
pointer2 has 77 in its stored address

step 3
pointer1 has 77 in its stored address
pointer2 has 14 in its stored address

step 4
pointer1 has 77 in its stored address
pointer2 has 21 in its stored address
```

ผลลัพธ์

```
#include <stdio.h>

int main ()
{
    int melon = 21;
    int banana = 14;

    int *ptr_one , *ptr_two ;

    ptr_one = &melon;
    ptr_two = ptr_one;

    printf("step 1\n");
    printf("pointer 1 has %d in its stored address\n", *ptr_one);
    printf("pointer 2 has %d in its stored address\n\n", *ptr_two);

    melon = 77;
    ptr_one = &melon;
    ptr_two = ptr_one;

    printf("step 2\n");
    printf("pointer 1 has %d in its stored address\n", *ptr_one);
    printf("pointer 2 has %d in its stored address\n\n", *ptr_two);

    ptr_two = &banana;

    printf("step 3\n");
    printf("pointer 1 has %d in its stored address\n", *ptr_one);
    printf("pointer 2 has %d in its stored address\n\n", *ptr_two);

    *ptr_two = 21;

    printf("step 4\n");
    printf("pointer 1 has %d in its stored address\n", *ptr_one);
    printf("pointer 2 has %d in its stored address", *ptr_two);
    return 0;
}
```

```
pointer 1 has 77 in its stored address
pointer 2 has 77 in its stored address

step 3
pointer 1 has 77 in its stored address
pointer 2 has 14 in its stored address

step 4
pointer 1 has 77 in its stored address
pointer 2 has 21 in its stored address
```

L0903 ให้นักศึกษาเขียนโปรแกรมเพื่อทดลองการสร้างและใช้งาน Pointer และ Function เบื้องต้น กำหนดให้มี Function ที่มี Function Prototype ดังนี้

```
float speedDistance(float speed, int *time);
```

โดย Function นี้ทำการคำนวณค่าระยะทาง (Distance) ด้วยสูตร $\text{Distance} = \text{Speed} \times \text{Time}$ แล้วให้ผลลัพธ์ออกมาเป็นค่า Distance ที่คำนวณได้ แต่ Function นี้จะมีการเพิ่มค่า Time ขึ้น 3 เมื่อคำนวณค่า Distance เสร็จสิ้นด้วย

ให้นักศึกษาเขียนโปรแกรมเพื่อเรียกใช้งาน Function นี้เป็นจำนวน 3 ครั้ง (ใช้ Loop ได้ตามสะดวก) โดยแต่ละครั้งที่เรียกใช้ Function ที่ชื่อ **speedDistance()** นี้ ค่า Speed ที่ใช้ ให้โปรแกรมทำการรับค่ามาจากผู้ใช้โปรแกรมผ่านทางคีย์บอร์ด เมื่อใช้งาน **speedDistance()** เสร็จสิ้นในแต่ละรอบ ให้พิมพ์ค่าของ Distance ที่คำนวณได้และค่า Time ออกมาด้วย

กำหนดให้ค่า Time เริ่มต้นที่ 1

ตัวอย่างผล

```
Distance calculator
-----
Enter speed: 15
step 1 => distance 15.00, time 4
Enter speed: 7
step 2 => distance 28.00, time 7
Enter speed: 9
step 3 => distance 63.00, time 10
```

```
Distance calculator
-----
Enter speed: 124.5
step 1 => distance 124.50, time 4
Enter speed: 42.3
step 2 => distance 169.20, time 7
Enter speed: 12.77
step 3 => distance 89.39, time 10
```

ผลลัพธ์

```
#include <stdio.h>
float speedDistance(float speed, int *time)
{
    float distance = speed * *time;
    *time = *time + 3;
    return distance;
}
int main ()
{
    float speed , distance;
    int time = 1;

    printf("Distance calculator\n");
    printf("-----\n");

    for(int i = 0 ; i < 3 ; i++)
    {
        printf("Enter speed: ");
        scanf("%f", &speed);
        distance = speedDistance(speed , &time);
        printf("step %d => distance %.2f, time %d\n", i + 1 , distance , time);
    }

    return 0;
}
```

```
Distance calculator
-----
Enter speed: 15
step 1 => distance 15.00, time 4
Enter speed: 7
step 2 => distance 28.00, time 7
Enter speed: 9
step 3 => distance 63.00, time 10
```

L0904 ให้นักศึกษาเขียนโปรแกรมเพื่อทดลองการสร้างและใช้งาน Pointer และ Array เบื้องต้น

กำหนดให้มี Array 1 ตัวที่สามารถเก็บข้อมูลเลขจำนวนเต็มได้ 12 ค่า

ให้นักศึกษาเขียนโปรแกรมเพื่อแสดงผลค่าผลคูณของสมาชิกแต่ละตัวใน Array กับ 24 โดยใช้ Pointer ในการช่วยชี้ไปที่สมาชิกแต่ละตัว

ให้นักศึกษาแสดงผลค่าเดิมของสมาชิกแต่ละตัวด้วย Pointer ประกอบไปด้วยเช่นกัน

ค่าของสมาชิกแต่ละตัวใน Array นักศึกษาสามารถกำหนดได้อย่างอิสระ ไม่จำกัดวิธีการในการกำหนดค่าให้ Array จะกำหนดโดยตรง กำหนดเป็นค่าเริ่มต้น หรือจะทำการรับค่ามาจากผู้ใช้ สามารถทำได้ทั้งหมด

ตัวอย่างผล

```
original ::: 4 5 6 7 8 9 10 11 12 13 14 15
multiplied ::: 96 120 144 168 192 216 240 264 288 312 336 360
```

ผลลัพธ์

```
#include <stdio.h>

int main ()
{
    int arr[12] = {4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15};
    int *ptr_arr;

    printf("original ::: ");
    for(int i = 0 ; i < 12 ; i++)
    {
        ptr_arr = &arr[i];
        printf("%d ", *ptr_arr);
    }
    printf("\nmultiplied ::: ");
    for(int i = 0 ; i < 12 ; i++)
    {
        ptr_arr = &arr[i];
        printf("%d ", *ptr_arr * 24);
    }
    return 0;
}
```

```
original ::: 4 5 6 7 8 9 10 11 12 13 14 15
multiplied ::: 96 120 144 168 192 216 240 264 288 312 336 360
```

L09001 ให้นักศึกษาเขียนโปรแกรมประยุกต์การใช้ความรู้จากหัวข้อเรื่อง Pointer

เขียนโปรแกรมจำลองการต่อสู้ระหว่างการ์ด 2 ใบคือการ์ดของผู้เล่นและคู่ต่อสู้ การ์ดแต่ละใบจะมีค่าพลังโจมตี (ATK) และพลังป้องกัน (DEF) การต่อสู้นั้น จะวัดกันที่ค่าพลัง ATK ของแต่ละฝ่าย ใคร ATK สูงกว่าจะชนะและทำลายอีกฝ่ายได้

แต่การ์ดของผู้เล่นมีคุณสมบัติพิเศษคือ สามารถสลับค่าพลังโจมตีและพลังป้องกันได้ โดยโปรแกรมจะทำการถามว่าต้องการจะสลับค่าเหล่านี้ของการ์ดผู้เล่นหรือไม่ ถ้าใช่ ให้สลับค่าทั้ง 2 ค่านี้

การสลับค่า ให้เขียนเป็น Function ที่มี Parameter เป็น Pointer 2 ตัวสำหรับรับ Address ของข้อมูลที่ต้องการจะสลับค่ากัน

ตัวอย่างผลลัพธ์

```
Card Battle!

Player
-----
ATK :: 1100      DEF :: 2500

Opponent
-----
ATK :: 800       DEF :: 1700

Switch player's ATK and DEF? (y/n): y

Player
-----
ATK :: 2500      DEF :: 1100

Attack calculating ATK vs ATK
Opponent destroyed!
```

```
Card Battle!

Player
-----
ATK :: 2300      DEF :: 3400

Opponent
-----
ATK :: 1400      DEF :: 200

Switch player's ATK and DEF? (y/n): n

Attack calculating ATK vs ATK
Opponent destroyed!
```

Card Battle!

Player

ATK :: 2500 DEF :: 600

Opponent

ATK :: 2300 DEF :: 1300

Switch player's ATK and DEF? (y/n): y

Player

ATK :: 600 DEF :: 2500

Attack calculating ATK vs ATK

Player destroyed!

Card Battle!

Player

ATK :: 800 DEF :: 1000

Opponent

ATK :: 600 DEF :: 300

Switch player's ATK and DEF? (y/n): x

Error input

Switch player's ATK and DEF? (y/n): n

Attack calculating ATK vs ATK

Opponent destroyed!

ผลลัพธ์

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void roll_stat(int stat[], int size, int *result)
{
    *result = stat[rand() % size];
}

void bot_roll_stat(int bot_stat[], int bot_size, int *bot_result)
{
    *bot_result = bot_stat[rand() % bot_size];
}

int main ()
{
    int stat[4] = {1100, 2300, 2000, 1000};
    int bot_stat[4] = {600, 1400, 1700, 2200};
    int size = sizeof(stat) / sizeof(stat[0]);
    int bot_size = sizeof(bot_stat) / sizeof(bot_stat[0]);
    char choice;
    int atk, def, bot_atk, bot_def;

    srand(time(NULL));

    roll_stat(stat, size, &atk);
    roll_stat(stat, size, &def);
    bot_roll_stat(bot_stat, bot_size, &bot_atk);
    bot_roll_stat(bot_stat, bot_size, &bot_def);
    printf("Card battle!\n\n");

    printf("Player\n");
    printf("-----");
    printf("ATK :: %d      DEF :: %d\n", atk, def);

    printf("Opponent\n");
    printf("-----");
    printf("ATK :: %d      DEF :: %d\n", bot_atk, bot_def);

    do
    {
        printf("Switch player's ATK and DEF (y/n): ");
        scanf(" %c", &choice);
        if(choice == 'y')
        {
            int temp = atk;
            atk = def;
            def = temp;
        }
        if(choice != 'y' && choice != 'n')
        {
            printf("ERROR input\n");
        }
    } while (choice != 'y' && choice != 'n');

    printf("Attack calculating ATK vs ATK\n");
    if(atk > bot_atk)
    {
        printf("Opponent destroyed!\n");
    }
    else
    {
        printf("Player destroyed!\n");
    }
    return 0;
}
```

```
Card battle!

Player
-----ATK :: 1100      DEF :: 2300

Opponent
-----ATK :: 2200      DEF :: 1400

Switch player's ATK and DEF (y/n): u
ERROR input

Switch player's ATK and DEF (y/n): y

Attack calculating ATK vs ATK
Opponent destroyed!
```

L08002 ให้นักศึกษาเขียนโปรแกรมประยุกต์การใช้ความรู้จากหัวข้อเรื่อง Pointer

ให้นักศึกษาเขียนเกมบันไดงู จำลองกระดานขนาด 5×5 (25 ช่อง)

ในเกมบันไดงูนี้ ผู้เล่นจะทำการทอยลูกเต๋าจำนวน 1 ลูก จากนั้นผู้เล่นจะทำการเดินไปเป็นจำนวนช่องตามที่ลูกเต๋ากำหนด (เช่น ตอนแรกอยู่ช่องที่ 1 ทอยลูกเต๋ได้ 3 ผู้เล่นจะเดินไปช่องที่ 4)

ถ้าผู้เล่นเดินตกลงไปยังช่องที่มีเลขลงท้ายด้วย 4 (4, 14, 24) ผู้เล่นจะต้องถอยหลังไปเป็นจำนวน 3 ช่อง
ถ้าผู้เล่นเดินถึงหรือเลยเส้นชัย (ช่องที่ 25) จะถือว่าจบเกม

ผู้เล่นจะเริ่มต้นเกมที่ช่องที่ 1 เสมอ

ช่องที่ผู้เล่นอยู่ ให้มีเครื่องหมายวงเล็บครอบ ช่องที่เป็นหลุมให้มีเครื่องหมาย _ ล้อม ในกรณีที่ผู้เล่นถึงเส้นชัยแล้ว ไม่ว่าจะถึงพอดีหรือเลยก็ตาม ให้วงเล็บครอบที่ช่องสุดท้าย (ช่อง 25)

Note: สามารถใช้ Pointer กับ Array ช่วยในการเดินและวาดกระดาน

ตัวอย่างผลลัพธ์

```
Snake and Ladder

Board :: (1) 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y): y
You got 2
Board :: 1 2 (3) _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25

Board :: 1 2 (3) _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y): y
You got 6
Board :: 1 2 3 _4_ 5 6 7 8 (9) 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25

Board :: 1 2 3 _4_ 5 6 7 8 (9) 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y): y
You got 1
Board :: 1 2 3 _4_ 5 6 7 8 9 (10) 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25

Board :: 1 2 3 _4_ 5 6 7 8 9 (10) 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y): y
You got 3
Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 (13) _14_ 15 16 17 18 19 20 21 22 23 _24_ 25

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 (13) _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y): y
You got 4
Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 (17) 18 19 20 21 22 23 _24_ 25

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 (17) 18 19 20 21 22 23 _24_ 25
Roll dice?(y): n
Error input

Roll dice?(y): y
You got 2
Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 (19) 20 21 22 23 _24_ 25

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 (19) 20 21 22 23 _24_ 25
Roll dice?(y): y
You got 6
Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ (25)

FINISH!
```

ผลลัพธ์

```

Board :: (1) 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y); y

Board :: (1) 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
You got 4

Board :: 1 2 3 _4_ (5) 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y); y

Board :: 1 2 3 _4_ (5) 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
You got 6

Board :: 1 2 3 _4_ 5 6 7 8 9 10 (11) 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y); y

Board :: 1 2 3 _4_ 5 6 7 8 9 10 (11) 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ 25
You got 5

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 (16) 17 18 19 20 21 22 23 _24_ 25
Roll dice?(y); y

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 (16) 17 18 19 20 21 22 23 _24_ 25
You got 2

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 (18) 19 20 21 22 23 _24_ 25
Roll dice?(y); y

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 (18) 19 20 21 22 23 _24_ 25
You got 5

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 (23) _24_ 25
Roll dice?(y); y

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 (23) _24_ 25
You got 3

Board :: 1 2 3 _4_ 5 6 7 8 9 10 11 12 13 _14_ 15 16 17 18 19 20 21 22 23 _24_ (25)
FINISH!

```